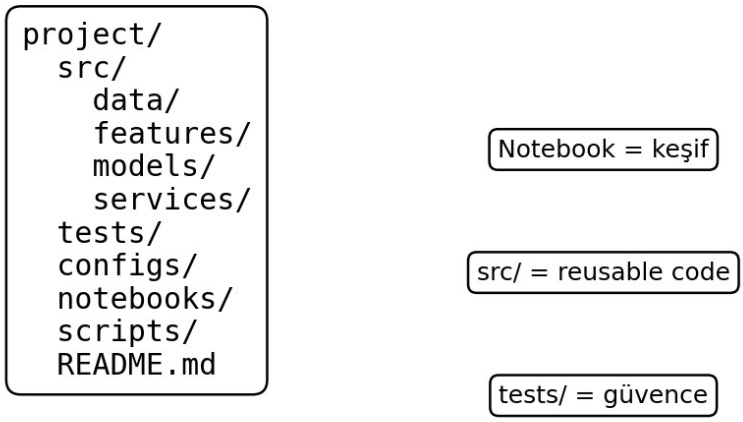
38. SOFTWARE ENGINEERING

Clean Code • Project Structure • Logging • Config • Tests • Type Hints • Packaging

Data science projesini üretim düzenine taşımak



Bu modülün amacı
Notebook'ta çalışan kodu güvenilir, test edilebilir, okunabilir ve tekrar kullanılabilir bir yazılım projesine dönüştürmek. Bir veri bilimci için software engineering; model geliştirmekten üretime geçiş arasındaki köprüdür.

Kazanım	Modül sonunda
Clean code	Okunabilir fonksiyonlar, isimlendirme ve sorumluluk sınırları kurabilirsin.
Project structure	Notebook, src, tests, config ve scripts ayrımını tasarlayabilirsin.
Reliability	Exceptions, logging ve validation ile failure'ları yönetebilirsin.
Testing	Unit/integration testleri ve fixtures kullanabilirsin.
Typing	Type hints, dataclass ve static checking mantığını anlayabilirsin.
Production	Packaging, config, dependency management ve observability temellerini kurabilirsin.

1. Software Engineering neden veri bilimci için gerekli?

Bir modelin notebook'ta doğru sonuç üretmesi tek başına yeterli değildir. Kod başkası tarafından çalıştırılabilmeli, test edilebilmeli, tekrar üretilebilmeli ve hata verdiğinde neden verdiği anlaşılabilmelidir.

Notebook problemi	Engineering karşılığı
Hücre sırasına bağımlı	Deterministic function/module
Hard-coded path	Config
print ile debug	Structured logging
Manuel kontrol	Automated tests
Global değişken	Explicit dependency
Tek dosya	Modular project

Ana fikir
Çalışan kod ile sürdürülebilir yazılım aynı şey değildir.

2. Clean code

- İyi isimlendirme
- Küçük ve tek sorumluluklu fonksiyonlar
- Az sürprizli API
- Tekrarı azaltma
- Yan etkileri görünür kılma

Clean code = kısa kod değil
Amaç kodun niyetini kolayca anlatmasıdır.

3. İsimlendirme

Kötü	Daha iyi
x	customer_features
df2	clean_orders
do_it()	build_daily_sales()
tmp	validated_batch
res	fraud_scores

İsim uzunluğu
Scope büyüdükçe daha açıklayıcı isim kullanmak genellikle daha iyidir.

4. Fonksiyon tasarımı

```
def calculate_revenue(
    quantity: int,
    unit_price: float
) -> float:
    return quantity * unit_price
```

- Tek iş yap.
- Input/output açık olsun.
- Global state'e mümkün olduğunca bağımlı olmasın.
- Yan etkileri sınırlı tut.

Pure function
Aynı input -> aynı output; dış state'i değiştirmez. Test etmesi kolaydır.

5. Fonksiyon boyutu

Fonksiyonun satır sayısından çok kavramsal sorumluluğu önemlidir.

Soru
Bu fonksiyonun adında 've' kullanmak zorunda kalıyorsan muhtemelen iki sorumluluk vardır.

6. DRY ve aşırı soyutlama

DRY - Don't Repeat Yourself - tekrar eden business logic'i tek yerde tutmayı amaçlar.

Ama dikkat
İki benzer satırı erken soyutlamak, aslında farklılaşacak iki davranışı gereksiz bağlayabilir. Önce gerçek tekrar oluşsun.

7. YAGNI

You Aren't Gonna Need It: henüz gerekmeyen esnekliği ve feature'ı önceden ekleme.

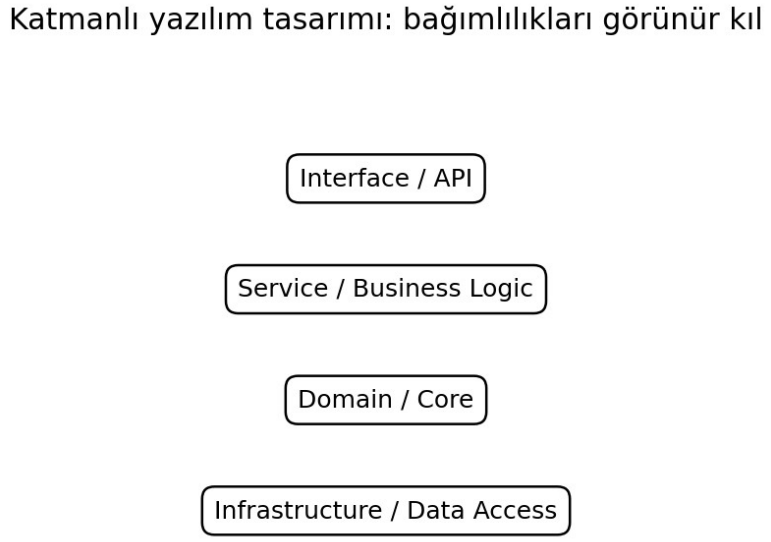
Data science pratiği
İlk model için 12 katmanlı plugin sistemi kurma; önce gerçek kullanım ortaya çıksın.

8. SOLID'e pratik giriş

Prensip	Veri bilimi yorumu
S - Single Responsibility	Feature builder yalnız feature üretsin
O - Open/Closed	Yeni model eklerken mevcut kodu az değiştir
L - Liskov	Aynı interface implementasyonları beklenen davranışı korusun
I - Interface Segregation	Büyük gereksiz interface yerine küçük sözleşmeler
D - Dependency Inversion	Concrete DB yerine repository/protocol abstraction

Ezber değil
SOLID'in amacı bağımlılıkları azaltmak ve değişikliği ucuzlatmaktır.

9. Katmanlı tasarım



Katman	Örnek
Interface	FastAPI endpoint / CLI
Service	Prediction workflow
Domain	Feature rules / business logic
Infrastructure	DB, file, cloud clients

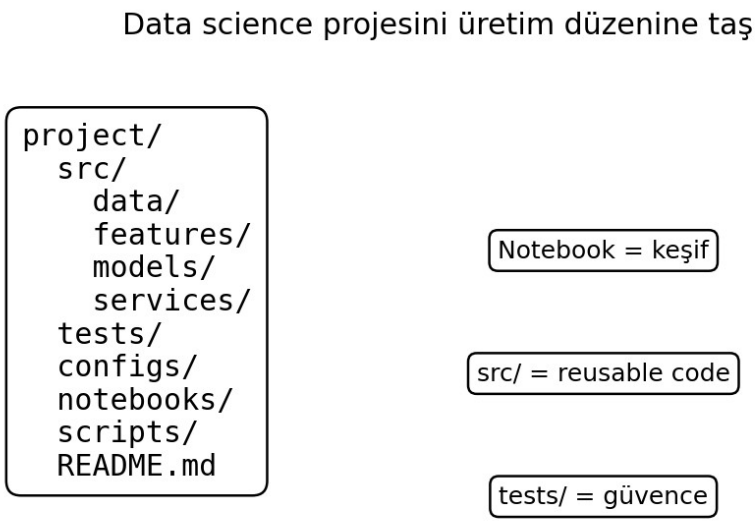
Bağımlılık yönü
Core logic mümkün olduğunca infrastructure detaylarını bilmemelidir.

10. Dependency injection

```
class FraudService:
    def __init__(self, model, repository):
        self.model = model
        self.repository = repository
```

Neden?
Testte gerçek database yerine fake repository verebilirsin.

11. Proje klasör yapısı




```
project/
  src/
    data/
    features/
    models/
    services/
  tests/
  configs/
  notebooks/
  scripts/
  pyproject.toml
  README.md
```

Notebook rolü
Keşif ve iletişim içindir; production business logic notebook hücrelerinde kalmamalıdır.

12. src layout

Reusable kodu import edilebilir package altında tutmak path sorunlarını ve test karmaşasını azaltır.

```
src/
  churn_project/
    __init__.py
    data.py
    features.py
    model.py
    service.py
```

13. __init__.py ve package

Python klasörünü package olarak düzenler ve public API'yi kontrol etmekte kullanılabilir.

Basit tut
Her şeyi __init__.py içine import etmeye çalışmak circular import riskini artırabilir.

14. Notebooks klasörü

- 01_eda.ipynb
- 02_features.ipynb
- 03_modeling.ipynb

Notebook hygiene
Kernel restart + run all ile baştan sona çalışmalı; gizli state olmamalı.

15. Configuration management

Model parametreleri, path'ler ve environment-specific ayarlar kod içine gömülmemelidir.

```
# config.yaml
data:
  train_path: data/train.parquet

model:
  learning_rate: 0.03
  max_depth: 6
```

Config ≠ secret
API key/password config dosyasına değil secret manager/environment variable'a konur.

16. Environment variables

```
import os

db_url = os.environ["DB_URL"]
```

Neden?
Dev/test/prod ortamları arasında deploy edilebilirliği artırır ve secret'ı source code'dan ayırır.

17. dataclass ile config

```
from dataclasses import dataclass

@dataclass(frozen=True)
class ModelConfig:
    learning_rate: float
    max_depth: int
```

Fayda
Typed, immutable config ile typo ve yanlış kullanım riski azalır.

18. Pydantic/validation mantığı

Runtime input veya config verisini schema'ya göre doğrulamak production sistemlerde önemlidir.

API bağlantısı
FastAPI gibi framework'lerde request validation aynı prensibe dayanır.

19. Type hints

```
def predict(
    features: list[float]
) -> float:
    ...
```

Tip	Örnek
Primitive	int, float, str, bool
Collection	list[str], dict[str, float]
Optional	str | None
Protocol	Interface-like structural typing
TypedDict	Dictionary schema

Type hint runtime garantisi değildir
Static checker ve IDE desteği sağlar; runtime validation ayrı konudur.

20. mypy / static checking

Type annotation'lar üzerinden bazı hataları kod çalışmadan bulur.

Örnek
Fonksiyon float beklerken dict gönderilmesi daha erken görünür.

21. Exceptions

```
class DataValidationError(Exception):
    pass

if df.empty:
    raise DataValidationError("Input dataset is empty")
```

Exception mesajı
Ne oldu + hangi input/state + kullanıcı ne yapmalı bilgisi vermeli.

22. try/except doğru kullanım

```
try:
    data = load_data(path)
except FileNotFoundError as exc:
    logger.exception(
        "Training data not found",
        extra={"path": path}
    )
    raise
```

Yutma
except Exception: pass üretimde en tehlikeli pattern'lerden biridir.

23. Fail fast

Hatalı input'u pipeline sonunda değil mümkün olduğunca erken reddet.

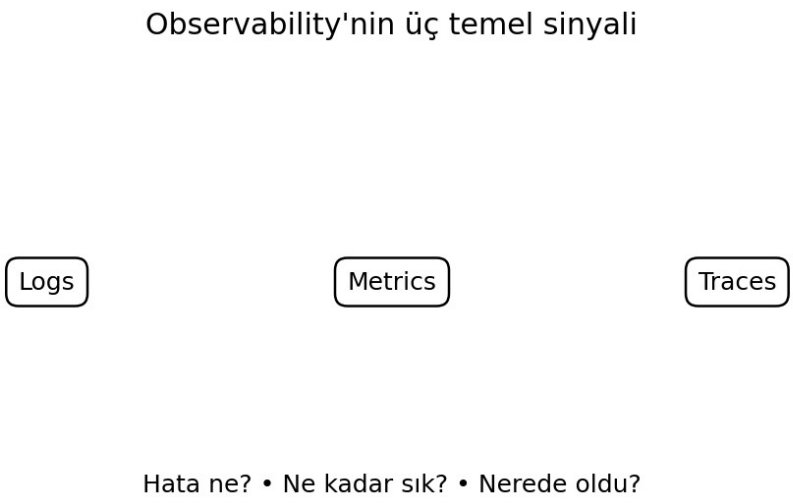
- Schema missing
- Empty dataset
- Invalid category
- Negative amount

Sessiz bozulma
Yanlış veriyle devam eden pipeline, açıkça fail olan pipeline'dan daha tehlikeli olabilir.

24. Assertions vs validation

Assert	Explicit validation
Developer invariant	User/data error
Optimization ile kapatılabilir	Production logic
Internal sanity	External input

25. Logging




```
import logging

logger = logging.getLogger(__name__)

logger.info(
    "training_started",
    extra={
        "rows": len(train_df),
        "model": "lightgbm"
    }
)
```

print yerine
Logging level, timestamp, module ve metadata ile production debugging'i mümkün kılar.

26. Log seviyeleri

Level	Kullanım
DEBUG	Detaylı development bilgisi
INFO	Normal önemli event
WARNING	Beklenmeyen ama devam edilebilir durum
ERROR	Operation başarısız
CRITICAL	Sistem seviyesinde ciddi problem

27. Structured logging

```
{
  "event": "prediction_completed",
  "model_version": "v12",
  "latency_ms": 48,
  "request_id": "abc123"
}
```

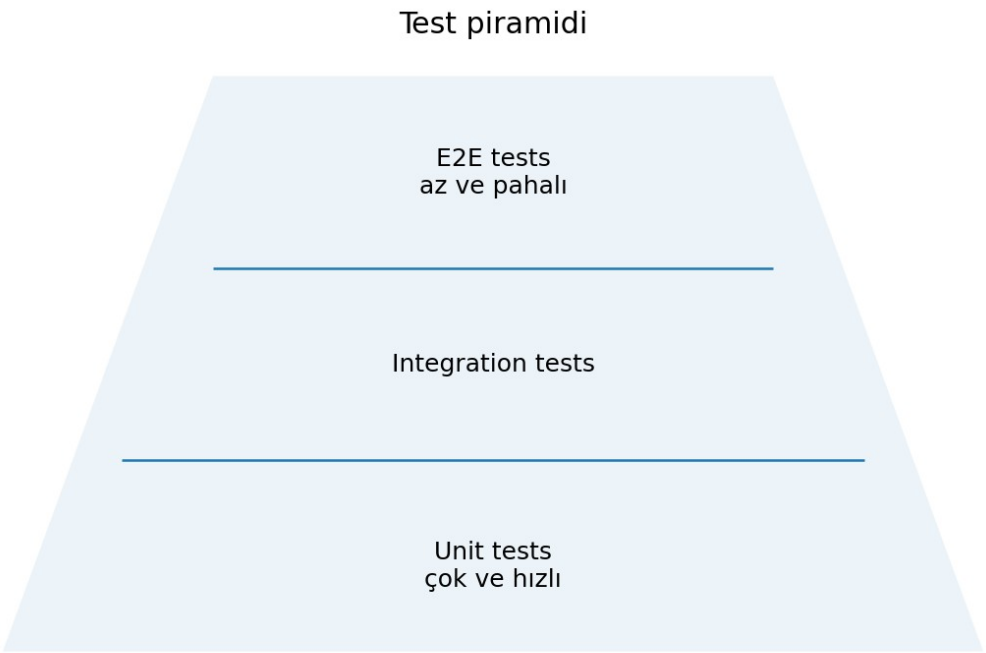
Neden JSON?
Log query, aggregation ve dashboard kurmak kolaylaşır.

28. Correlation / request ID

Dağıtık sistemde aynı request'in API, feature service ve model endpoint loglarını bağlayan kimlik.

Debug
Bir kullanıcı şikayetini tek request_id ile uçtan uca izleyebilirsin.

29. Unit testing




```
def test_revenue():
    assert calculate_revenue(
        quantity=3,
        unit_price=10.0
    ) == 30.0
```

Unit test
Küçük bir fonksiyonu external dependency olmadan hızlı test eder.

30. pytest

- Simple assert
- Fixtures
- Parametrize
- Markers
- Exception testing

Hedef
Testlerin geliştiricinin sürekli çalıştırabileceği kadar hızlı olması.

31. Fixtures

```
import pytest
import pandas as pd

@pytest.fixture
def sample_orders():
    return pd.DataFrame(
        {
            "amount": [10, 20],
            "customer_id": [1, 2]
        }
    )
```

Fixture
Tekrar kullanılan test input/setup kodunu merkezi hale getirir.

32. Parametrized tests

```
@pytest.mark.parametrize(
    "x, expected",
    [
        (0, 0),
        (1, 1),
        (2, 4),
    ]
)
def test_square(x, expected):
    assert x**2 == expected
```

33. Edge-case testing

- Empty input

- Null values
- Unknown category
- Extreme numeric values
- Duplicate IDs
- Timezone boundary

Model pipeline
Happy path tek başına yeterli değildir; gerçek production bug'ları edge case'lerde çıkar.

34. Integration tests

Birden fazla component'in birlikte çalışmasını doğrular.

- Data loader + transform
- Model + feature builder
- API + fake database

Unit'ten pahalı
Bu yüzden sayısı daha az, kapsamı daha geniş tutulur.

35. End-to-end tests

Kullanıcıya en yakın bütün workflow'u test eder: request -> service -> model -> response.

Az ama kritik
Yavaş ve fragile olabilir; en önemli business flow'ları kapsamalı.

36. Mock / fake

External dependency'yi test sırasında kontrollü substitute ile değiştirmek.

Fake tercih
Mümkün olduğunda davranışa yakın in-memory fake, aşırı mock kullanımından daha anlaşılır olabilir.

37. Dataframe testleri

```
def test_clean_orders():
    out = clean_orders(input_df)

    assert out["order_id"].is_unique
    assert out["amount"].ge(0).all()
```

Sadece satır sayısı yetmez
Schema, key uniqueness, values ve deterministic output test et.

38. Model tests

Test	Amaç
Shape	Predict output beklenen boyutta mı?
Determinism	Fixed seed ile reproducible mı?
Serialization	Save/load sonrası tahmin aynı mı?
Smoke	Küçük batch ile crash olmuyor mu?
Regression	Metric kabul edilen eşikten aşağı düşmedi mi?

39. Golden test / regression test

Bilinen input'lar için beklenen output veya tolerance saklanır; refactor sonrası davranışın değişmediği doğrulanır.

ML caution
Model retraining bilinçli olarak output'u değiştiriyorsa golden set versiyonlanmalı.

40. Property-based testing

Tek örnek yerine input uzayındaki genel özellikleri test eder.

- Probability 0-1 arasında
- Sorted output gerçekten sıralı
- Normalization finite değer üretir

Araç mantığı
Hypothesis benzeri framework'ler edge case üretimini otomatikleştirir.

41. Dependency management

Projenin hangi package ve versiyonlara ihtiyaç duyduğunu tanımlar.

```
# pyproject.toml mantığı
[project]
dependencies = [
  "pandas>=2",
  "scikit-learn>=1.5"
]
```

requirements freeze
Application deployment için tam versiyon pinlemek reproducibility'yi artırabilir.

42. pyproject.toml

Modern Python projelerinde package metadata, dependencies, build ve tool config için merkezi dosyadır.

- project metadata
- dependencies
- pytest config
- formatter/linter config

43. Virtual environments

```
python -m venv .venv
source .venv/bin/activate
pip install -e .
```

Amaç
Projelerin dependency'lerini birbirinden izole etmek.

44. Editable install

pip install -e . ile package source üzerinde değişiklik yaparken yeniden kurulum gerekmeden import edebilirsin.

Notebook faydası
sys.path hacklemek yerine gerçek package import edilir.

45. Linting ve formatting

Araç sınıfı	Amaç
Formatter	Otomatik style
Linter	Potential bugs / style issues
Import sorter	Import düzeni
Static type checker	Type consistency

Tartışmayı otomatikleştir
Kod formatını ekip tartışması olmaktan çıkar; tool belirlesin.

46. Pre-commit

Commit oluşmadan önce formatter, linter, secret check gibi otomatik hook'lar çalıştırır.

37. modül bağlantısı
Git workflow'u ile code quality kontrolleri birleştirilir.

47. Documentation

- README
- Docstrings
- Architecture notes
- API usage examples
- Runbook

İyi dokümantasyon
Kodun ne yaptığından çok neden böyle yapıldığını anlatmalıdır.

48. Docstrings

```
def build_features(df):
    '''
    Build customer-level 30-day features.

    Parameters
    ----------
    df:
        Order-level input.

    Returns
    -------
    DataFrame
        One row per customer.
    '''
```

Kritik bilgi
Input grain, output grain, assumptions ve side effects.

49. Comments

Kötü comment	İyi comment
x = x + 1 # add one	# Offset required by source system's 1-based IDs
# loop	Why this loop cannot be vectorized safely

Kod ne yaptığını; comment neden yaptığını anlatsın
Açıklayıcı isim comment ihtiyacını azaltır.

50. Refactoring

Dış davranışı değiştirmeden iç tasarımı iyileştirmek.

- Extract function
- Rename
- Move module

- Remove duplication
- Introduce abstraction

Test güvenliği
Refactor öncesi tests varsa değişiklik daha güvenli yapılır.

51. Technical debt

Kısa vadede hız için alınan kararların gelecekte bakım maliyetine dönüşmesi.

Debt	Örnek
Code debt	Copy-paste feature logic
Test debt	Critical pipeline testsiz
Data debt	Schema undocumented
Infra debt	Manual deployment
ML debt	Feature train-serving mismatch

Borç olabilir
Ama bilinçli, görünür ve ödeme planı olan teknik borç yönetilebilir.

52. Code review

- Correctness
- Readability
- Tests
- Security
- Performance
- Backward compatibility

Review kişi değil kod içindir
Somut, açıklayıcı ve çözüm odaklı feedback verilmelidir.

53. API design

```
def predict_customer_churn(
    customer_id: str
) -> float:
    ...
```

İyi API
Az parametre, net isim, stabil contract ve açık hata davranışı.

54. Backward compatibility

Bir public function/API değişikliği downstream kullanıcıları kırmamalıdır.

- Default parametre
- Deprecation
- Versioned endpoint

Data contracts
Software API contract ve data schema contract aynı temel fikre dayanır.

55. Performance engineering

Önce ölç, sonra optimize et.

- Profiler
- Latency
- CPU
- Memory
- I/O

Premature optimization
Kanıt olmadan karmaşık optimizasyon yapmak readability'yi bozabilir.

56. Caching

Pahalı ve tekrar kullanılan sonucu saklamak.

Cache	Örnek
In-memory	Feature lookup
Disk	Processed artifacts
Distributed	Shared service cache

Invalidation
Cache'in en zor kısmı verinin ne zaman geçersiz olduğudur.

57. Serialization

- JSON
- Parquet
- Pickle/joblib
- ONNX

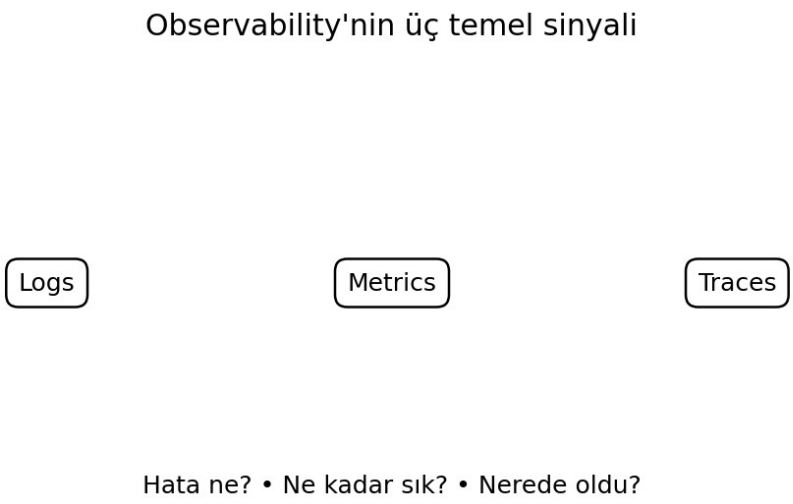
Güvenlik
Untrusted pickle dosyasını yüklemek risklidir; arbitrary code execution ihtimali vardır.

58. Versioning

Neyi versionla?	Örnek
Code	Git commit
Data	Snapshot/version
Model	model:v17
Config	YAML commit
Schema	Contract version

Reproducibility
Bir prediction için mümkünse code + data + model + config versiyonu bulunabilmeli.

59. Observability



Sinyal	Soru
Logs	Ne oldu?
Metrics	Ne kadar oldu?
Traces	Hangi adımda oldu?

ML sistemi
Bunlara model drift/performance ve data quality sinyalleri de eklenir.

60. Health checks

Servisin process olarak ayakta olması ile gerçekten prediction yapabilmesi farklıdır.

- Liveness

- Readiness

Readiness
Model yüklenmiş mi, dependencies erişilebilir mi? Traffic almadan önce bunu doğrula.

61. Graceful degradation

Dependency bozulduğunda sistemin tamamen çökmesi yerine güvenli fallback kullanması.

Örnek
Recommendation model yoksa popularity baseline döndür.

62. Timeouts ve retries

Pattern	Kural
Timeout	External call sonsuza kadar beklemesin
Retry	Geçici hatalarda sınırlı tekrar
Backoff	Retry aralığını artır
Jitter	Thundering herd azalt

Retry her yerde değil
Validation hatası veya 4xx tipi deterministic failure retry edilmemelidir.

63. Idempotency

Aynı request/job tekrar çalıştığında sonucu istenmeyen şekilde çoğaltmama özelliği.

Örnek
Payment/feature materialization/batch scoring retry-safe olmalı.

64. Concurrency temeli

Aynı anda birden fazla işin yürütülmesi shared state ve race condition riskleri getirir.

- Thread/process
- Async I/O
- Locks
- Queues

Python DS pratiği
CPU-bound model scoring ile I/O-bound API çağrısının concurrency ihtiyaçları farklıdır.

65. Security basics

- Secrets source code'a girmez.
- Input validate edilir.
- Least privilege uygulanır.
- Dependency vulnerabilities izlenir.
- Sensitive data loglanmaz.

PII logging
Email, token, password, health/financial identifiers gibi hassas verileri loglama.

66. Dependency vulnerabilities

Third-party packages güvenlik açığı içerebilir; dependency scanning ve kontrollü güncelleme gerekir.

Lockfile
Hangi dependency versiyonunun deploy edildiğini bilmek incident response için kritiktir.

67. CI temel akışı

```
on pull_request:
  - install dependencies
  - lint
  - type check
  - run tests
  - build package
```

38 + 37 bağlantısı
Software engineering kontrolleri GitHub Actions gibi CI sistemleriyle her PR'da otomatik uygulanır.

68. Release mantığı

- Version
- Changelog
- Artifact
- Deployment
- Rollback

Rollback planı
Yeni sürüm sorun çıkarırsa önceki çalışan versiyona hızlı dönebilmelisin.

69. Semantic versioning

Versiyon	Anlam
MAJOR	Breaking change

MINOR	Backward-compatible feature
PATCH	Backward-compatible fix

Internal ML
Her ekip semver kullanmak zorunda değil; önemli olan açık version policy.

70. Data science anti-patterns

- Notebook'u production job olarak doğrudan çalıştırmak.
- Aynı preprocessing'i üç dosyada copy-paste etmek.
- Hard-coded absolute path kullanmak.
- print ile production debug yapmak.
- Exceptions'ı tamamen yutmak.
- Test yazmadan büyük refactor yapmak.
- Global mutable state kullanmak.
- Model ve feature config'ini koda gömmek.
- Random seed/version bilgisini kaydetmemek.
- sys.path.append ile package problemlerini gizlemek.

71. Notebook'tan production'a dönüşüm

Notebook aşaması	Production karşılığı
EDA cell	Reusable transform
Manual path	Config
Inline model	model.py
print metric	logger/metrics
Manual check	pytest
Kernel state	Explicit function inputs
Saved .pkl	Versioned artifact

72. Refactor örneği

```
# notebook
df["revenue"] = (
    df["quantity"] * df["price"]
)

# production
def add_revenue(df):
    required = {"quantity", "price"}
    missing = required - set(df.columns)

    if missing:
        raise DataValidationError(
            f"Missing columns: {missing}"
        )

    out = df.copy()
    out["revenue"] = (
        out["quantity"] * out["price"]
    )
    return out
```

Fark
Production versiyonu input contract, copy semantics ve failure behavior'ı açık hale getirir.

73. ML service örnek yapısı

```
src/churn/
  config.py
  features.py
  model.py
  repository.py
  service.py
  schemas.py
  logging.py

tests/
  test_features.py
  test_service.py
  test_model_io.py
```

74. Definition of Done

1. Kod review edildi.
2. Tests geçti.
3. Type/lint kontrolleri temiz.
4. Docs güncel.
5. Config hard-code değil.
6. Logs yeterli.
7. Error handling tanımlı.
8. Rollback planı var.

75. Mülakat soruları

Soru	Kısa cevap
Clean code nedir?	Niyeti açık, test edilebilir, sürdürülebilir kod.
Pure function?	Aynı input aynı output; dış state side effect yok.
Dependency injection?	Dependency'yi içeride yaratmak yerine dışarıdan vermek.
SOLID neden?	Değişiklik maliyeti ve coupling'i azaltmak.
Unit vs integration test?	Tek component vs component'lerin birlikte davranışı.
Fixture?	Tekrar kullanılan test setup/input.
Mock vs fake?	Mock interaction odaklı; fake çalışan basit implementasyon.
Type hint faydası?	IDE/static analysis ve interface netliği.
Fail fast?	Invalid state'i erken reddetmek.
Structured logging?	Query edilebilir metadata içeren log.
Idempotency?	Retry sonrası duplicate/bozuk sonuç üretmemek.
Timeout neden?	External dependency sonsuz beklemeyi önlemek.
Semantic versioning?	Major/minor/patch change policy.
Config vs secret?	Config davranış ayarı; secret credential.
Technical debt?	Kısa vadeli kararın gelecekte bakım maliyeti.

76. Uygulama alıştırmaları

9. Eski bir notebook'taki preprocessing kodunu functions.py modülüne taşı.
10. Hard-coded path'leri YAML config'e çıkar.
11. 3 fonksiyon için type hints ve docstrings ekle.
12. Custom DataValidationError yaz.
13. print çağrılarını logging ile değiştir.
14. pytest ile 10 unit test yaz.
15. Fixture ve parametrize kullan.
16. Model serialization regression testi yaz.
17. pyproject.toml oluştur ve editable install yap.
18. pre-commit içinde formatter + linter + test smoke check tasarla.

77. Mini portföy projesi

Proje: Production-Quality ML Python Package

Dosya	İçerik
src/project/config.py	Typed config
src/project/data.py	Input validation/load
src/project/features.py	Reusable features
src/project/model.py	Train/save/load
src/project/service.py	Prediction workflow
src/project/logging.py	Structured logging setup
tests/	Unit + integration tests
pyproject.toml	Package/dependencies/tools
.github/workflows/ci.yml	Lint + test + build
README.md	Architecture + run instructions

CV cümlesi
Refactored an ML workflow into a production-quality Python package with typed configuration, modular services, structured logging, custom validation, automated unit/integration tests, dependency management and CI checks.

78. Final cheat sheet

Kavram	Hatırlanacak fikir
Clean code	Niyeti açık kod
SRP	Bir component bir ana sorumluluk
Dependency injection	External dependency dışarıdan gelsin
Config	Davranış koddan ayrı
Secret	Credential source code dışında
Type hints	Static contract
Exception	Failure açık ve anlamlı
Logging	Production görünürlüğü
Unit test	Küçük hızlı test
Integration test	Bileşenleri birlikte test
Fixture	Reusable test setup
Refactor	Davranışı koruyarak tasarım iyileştirme
pyproject	Modern package config
Pre-commit	Commit öncesi otomatik kalite
CI	Her değişiklikte otomatik kontrol
Idempotency	Retry-safe behavior
Timeout	Sonsuz beklemeyi önle
Observability	Logs + metrics + traces
Versioning	Code/data/model/config lineage

Bu modülden sonra
39. API Development bölümünde bu mühendislik prensiplerini FastAPI, REST, JSON schema, validation ve model-serving endpoint'lerine uygulayacaksın.